```
"""
uma/pipeline.py — The complete UMA pipeline. All 15 modules integrated.

FLOW (sphere outward, then field inward):

    AcousticSphereGeometry          sphere/geometry.py
        → SystemGeometry            (all params derived, nothing chosen)
        → SphereVenturi             sphere/field.py
        → SpherePendulum            sphere/field.py
        → SphereProjectionField     sphere/field.py
        → Planck input B(ν,T)       (e already structural)

    UMAClient + GENERICDynamics     client.py + dynamics/generic.py
        → H[ψ], dH/dψ*, S[ψ]       (actual Hamiltonian, not |z|²/2)
        → dissipative_drift         (entropy production)
        → psi_hat = -dH/dψ*         (MSR response field, exact)

    CrossDomainInjector             venturi/injector.py
        → VenturiOperator           venturi/operator.py
        → 360° azimuthal injection  (both domains: MSR + semantic)

    SemanticEngine                  semantic/engine.py
        → UMA_IR compiler           semantic/ir.py
        → UMAExecutor               semantic/executor.py
        → SemanticFriction          semantic/friction.py
            (dE_dt convergence, calibrated thresholds)
        → ConstraintSet             semantic/constraints.py
            (Alpha/Beta/Gamma anchors)
        → Inarticulator             semantic/inarticulation.py

    MSR verification                msr/stress_energy.py
        → <T_μν> ~ G_μν^(1)        (GR confirmed per step)

    Wetterich RG                    msr/wetterich_flow.py
        → universality class        (Lévy basin, z=1.5)

OUTPUT: UMAPipelineResult
    All geometry (exact, derived)
    H[ψ] trajectory (real Hamiltonian, not proxy)
    Friction walk (Bellman convergence)
    Pendulum samples (sphere field measurements)
    MSR/GR cosine similarity
    RG basin classification
    Closure status
```

```
"""

import math
import numpy as np
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional

from uma import Config, UMAClient
from uma.core.state import FieldPosterior
from uma.observations.base import GaussianObservation

from uma.sphere.geometry import AcousticSphereGeometry, SystemGeometry
from uma.sphere.field import SphereProjectionField, SpherePendulum, SphereVenturi

from uma.venturi.operator import VenturiOperator, VenturiConfig
from uma.venturi.injector import CrossDomainInjector

from uma.semantic.engine import SemanticEngine, SemanticEngineConfig
from uma.semantic.friction import SemanticFriction, SemanticFrictionConfig, Frict
from uma.semantic.constants import CALIBRATED_E_TARGET, CALIBRATED_DZ_DT_FLOOR

from uma.msr.stress_energy import verify_T_equals_lichnerowicz
from uma.msr.tensor_bridge import TensorBridge
from uma.msr.wetterich_flow import LevyMSRCouplings, classify_basin, dynamic_expo


# ---------------------------------------------------------------------------
# Physical constants
# ---------------------------------------------------------------------------

H_PLANCK = 6.626e-34
K_BOLTZ  = 1.381e-23
C_LIGHT  = 3e8
C_AIR    = 343.0


# ---------------------------------------------------------------------------
# Result
# ---------------------------------------------------------------------------

@dataclass
class UMAPipelineResult:
    # Sphere geometry (all exact)
    geometry: SystemGeometry

    # Field trajectory
    H_trajectory: List[float]        # actual H[ψ] per step, not |z|²/2
```

```
    S_trajectory: List[float]        # entropy S[ψ] per step
    z_trajectory: List[np.ndarray]   # projected state
    E_proxy: List[float]             # |z|²/2 (proxy, for reference)

    # Pendulum field samples
    pendulum_samples: List[dict]

    # Semantic engine output
    friction_walk: List[float]
    is_closed: bool
    closure_step: Optional[int]
    stage: str

    # MSR/GR verification
    msr_cosine: float                # <T_μν> ~ G_μν^(1) similarity
    msr_verified: bool

    # Wetterich RG
    rg_basin: str                    # 'levy' or 'gaussian'
    rg_z: float                      # dynamic exponent

    # Interference
    interference_amplitudes: List[float]
    tensor_records: List[Any]      # TensorRecord per step

    # Summary
    n_steps: int
    dt: float

    def report(self):
        geo = self.geometry
        print("=" * 65)
        print("  UMA PIPELINE — COMPLETE RESULT")
        print("=" * 65)
        print()
        print("SPHERE GEOMETRY (derived inside-out, nothing chosen):")
        print(f"  Bessel zero j_nl:    {geo.mode.bessel_zero:.6f}  (= π exactly)"
        print(f"  Venturi r₀:          {geo.r0:.6f} m  = R·j_nl/π")
        print(f"  Bernoulli gain G:    {geo.G:.6f}")
        print(f"  Fan frequency:       {geo.f_fan:.4f} Hz")
        print(f"  Resonant T★:         {geo.T_star:.2f} K  (stellar temperature)"
        print(f"  Pendulum length:     {geo.L_pendulum:.6e} m")
        print(f"  Pendulum period:     {geo.T_pendulum:.6e} s")
        print()
        print("FIELD DYNAMICS (actual Hamiltonian H[ψ]):")
        H = np.array(self.H_trajectory)
        S = np.array(self.S_trajectory)
```

```
print(f"  H initial:  {H[0]:.6e}")
print(f"  H final:    {H[-1]:.6e}")
print(f"  H mean:     {H[-30:].mean():.6e}")
print(f"  S initial:  {S[0]:.4f}  (entropy)")
print(f"  S final:    {S[-1]:.4f}")
print(f"  dS/step:    {(S[-1]-S[0])/self.n_steps:.6e}  (entropy productio
print()
print("BELLMAN CONVERGENCE (semantic friction):")
fw = self.friction_walk
step_size = max(1, len(fw)//8)
print(f"  Walk: {[round(fw[i],3) for i in range(0,len(fw),step_size)]}")
print(f"  Stage:    {self.stage}")
print(f"  Closed:   {self.is_closed}  (Ω achieved)")
if self.closure_step:
    print(f"  At step:  {self.closure_step}")
print()
print("MSR = EINSTEIN TENSOR (GR verification):")
print(f"  cos(<T_μν>, G_μν^(1)) = {self.msr_cosine:.6f}")
print(f"  Verified: {self.msr_verified}  (|cos|=1 → proportional)")
print()
print("WETTERICH RG FLOW:")
print(f"  Basin: {self.rg_basin}  (universality class)")
print(f"  Dynamic exponent z = {self.rg_z:.2f}")
print(f"  {'Non-local Lévy dynamics' if self.rg_basin=='levy' else 'Local
print()
print("SPHERE PENDULUM:")
Ps = [s['P_abs'] for s in self.pendulum_samples]
print(f"  P mean:  {np.mean(Ps):.6f}  (standing wave pressure)")
print(f"  P range: [{min(Ps):.4f}, {max(Ps):.4f}]")
print()
print("INTERFERENCE (starlight ⊗ synthetic):")
A = self.interference_amplitudes
print(f"  A mean:  {np.mean(A):.6e}")
print(f"  A range: [{min(A):.4e}, {max(A):.4e}]")
print()
print("TENSOR BRIDGE (T_μν ↔ G_μν^(1) live residual):")
if self.tensor_records:
    residuals = [r.relative_residual for r in self.tensor_records]
    kappas    = [r.kappa for r in self.tensor_records]
    satisfied = [r.einstein_satisfied for r in self.tensor_records]
    final_r   = self.tensor_records[-1]
    print(f"  κ (proportionality):  {final_r.kappa:.6f}")
    print(f"  Residual initial:     {residuals[0]:.6f}")
    print(f"  Residual final:       {residuals[-1]:.6f}")
    print(f"  Residual min:         {min(residuals):.6f}")
    print(f"  Einstein satisfied:   {final_r.einstein_satisfied}")
    n_sat = sum(satisfied)
```

```
            print(f"  Steps satisfied:      {n_sat}/{len(satisfied)}")
            print(f"  T_μν final:")
            print(f"    [[{final_r.T[0,0]:+.4e}, {final_r.T[0,1]:+.4e}]")
            print(f"     [{final_r.T[1,0]:+.4e}, {final_r.T[1,1]:+.4e}]]")
            print(f"  G_μν^(1) final:")
            print(f"    [[{final_r.G[0,0]:+.4e}, {final_r.G[0,1]:+.4e}]")
            print(f"     [{final_r.G[1,0]:+.4e}, {final_r.G[1,1]:+.4e}]]")
        print("=" * 65)


# ---------------------------------------------------------------------------
# Pipeline
# ---------------------------------------------------------------------------

class UMAPipeline:
    """
    The complete UMA pipeline. All 15 modules integrated.

    Usage::
        pipe = UMAPipeline(L=1.0, n_steps=200)
        result = pipe.run()
        result.report()
    """

    def __init__(
        self,
        L: float = 1.0,          # box side length (m)
        n_mode: int = 0,
        l_mode: int = 0,
        mode_index: int = 1,
        n_steps: int = 200,
        dt: float = 0.04,
        seed: int = 42,
        verbose: bool = True,
        input_text: str = "THRUPUT",
    ):
        self.L = L
        self.n_mode = n_mode
        self.l_mode = l_mode
        self.mode_index = mode_index
        self.n_steps = n_steps
        self.dt = dt
        self.seed = seed
        self.verbose = verbose
        self.input_text = input_text

    def _log(self, msg):
```

```
    if self.verbose:
        print(f"  {msg}")

# ---------------------------------------------------------------------
# Planck / interference
# ---------------------------------------------------------------------

def _planck_normalized(self, nu: float, T: float) -> float:
    """B(ν,T) / B(ν_peak, T) — normalized to peak = 1."""
    x     = H_PLANCK * nu / (K_BOLTZ * T)
    x_pk  = 2.821  # Wien peak
    B     = x**3 / (math.exp(x) - 1 + 1e-30)
    B_pk  = x_pk**3 / (math.exp(x_pk) - 1)
    return float(B / B_pk)

def _interference_amplitude(self, t: float, geo: SystemGeometry,
                            field_scale: float) -> float:
    """
    A(t) = field_scale × B_norm(ν_peak, T★) × cos(φ★ - φ_syn(t))

    φ★   = hν_peak / kT★  (Planck phase, e is already inside)
    φ_syn = ω_fan × t     (fan-modulated synthetic)
    """
    nu_peak    = geo.nu_star
    T_star     = geo.T_star
    omega_fan  = 2 * math.pi * geo.f_fan
    phi_star   = H_PLANCK * nu_peak / (K_BOLTZ * T_star)
    phi_syn    = omega_fan * t
    B_norm     = self._planck_normalized(nu_peak, T_star)
    return field_scale * B_norm * math.cos(phi_star - phi_syn)

# ---------------------------------------------------------------------
# Build injection field from amplitude
# ---------------------------------------------------------------------

def _injection_field(self, psi: np.ndarray, A: float,
                     rng: np.random.Generator) -> np.ndarray:
    """
    360° azimuthal injection field with amplitude A.
    Phase is uniformly distributed — no preferred direction.
    """
    N = psi.shape[0]
    theta = 2 * math.pi * rng.random((N, N))
    return A * np.exp(1j * theta)

# ---------------------------------------------------------------------
# Main run
```

```
# ------------------------------------------------------------------

def run(self) -> UMAPipelineResult:
    cfg  = Config()
    rng  = np.random.default_rng(self.seed)
    N    = cfg.grid.N
    kT   = cfg.generic.kT

    # ── 1. Sphere geometry (inside out) ──────────────────────────
    self._log("Solving sphere geometry (inside out)...")
    geo = AcousticSphereGeometry(
        L=self.L, n=self.n_mode, l=self.l_mode,
        mode_index=self.mode_index
    ).solve()

    if self.verbose:
        print()
        print(f"  Sphere: j_nl={geo.mode.bessel_zero:.4f}  "
              f"r0={geo.r0:.4f}m  f={geo.mode.f:.1f}Hz  "
              f"T★={geo.T_star:.1f}K  G={geo.G:.4f}")

    # Field scale: sqrt(2 * E_target)
    field_scale = math.sqrt(2 * CALIBRATED_E_TARGET)

    # ── 2. Sphere field + pendulum ────────────────────────────────
    sphere_field = SphereProjectionField(geo)
    pendulum     = SpherePendulum(geo)
    sphere_v     = SphereVenturi(geo)

    # ── 3. Wetterich RG (once — universality class) ──────────────
    c0       = LevyMSRCouplings(
        nu_2=cfg.generic.alpha, nu_alpha=0.1,
        D=kT*cfg.generic.lam, c_alpha=0.05, alpha=1.5, D_dim=2
    )
    rg_basin = classify_basin(c0)
    rg_z     = dynamic_exponent(c0)
    self._log(f"RG basin: {rg_basin}  z={rg_z:.2f}")

    # ── 4. MSR = Einstein (once) ──────────────────────────────────
    h_pert   = 0.01 * np.array([[1.0, 0.0],[0.0,-1.0]])
    msr_res  = verify_T_equals_lichnerowicz(h_pert, m=0.1, N=N)
    msr_cos  = float(msr_res["cosine_similarity"])
    msr_ok   = abs(abs(msr_cos) - 1.0) < 0.01
    self._log(f"MSR ~ Einstein: cos={msr_cos:.4f}  verified={msr_ok}")

    # ── 5. UMA client ─────────────────────────────────────────────
    self._log("Initializing UMA client...")
```

```
client = UMAClient(cfg)
psi0   = 0.3 * (rng.standard_normal((N,N)) +
                1j*rng.standard_normal((N,N)))
client.initialize(psi0, sigma0=0.05, dt=self.dt)
dim    = len(client.posterior_state.mean)

# z_target: let the field burn in 50 steps then use mean
self._log("Computing self-consistent z_target (50-step burn-in)...")
for _ in range(50):
    psi  = client.projection.lift(client.posterior_state.mean)
    psi  = client.dynamics.step(psi, self.dt, rng=rng)
    z    = client.projection.project(psi)
    client.posterior_state = FieldPosterior.full(
        z, client.posterior_state.Sigma)
z_target = client.posterior_state.mean.copy()
self._log(f"z_target={z_target}  |z*|={np.linalg.norm(z_target):.6f}")

# ── 6. Semantic engine ──────────────────────────────────────
friction_cfg = SemanticFrictionConfig(
    kT=kT, lam=cfg.generic.lam, N=N, dt=self.dt,
    convergence_dz_dt=CALIBRATED_DZ_DT_FLOOR,
    min_steps_before_closure=10,
    step_weight=0.025,
)
engine_cfg = SemanticEngineConfig(
    friction=friction_cfg,
    stop_on_closure=True,
    verbose=False,
    obs_every=2,
)
engine = SemanticEngine(
    z_target=z_target,
    config=engine_cfg,
)

# Observation: pendulum field sample projected to z-space
def make_obs(t: float) -> tuple:
    sample = pendulum.sample_field(sphere_field, t)
    y = np.full(dim, sample["P_real"] * field_scale / max(dim,1))
    R = (geo.r_in ** 2) * np.eye(dim)
    obs = GaussianObservation(
        output_dim=dim, R=R, H=np.eye(dim),
        h_fn=lambda z: z, h_jac=lambda z: np.eye(len(z))
    )
    return obs, y

# ── 7. Main loop ────────────────────────────────────────────
```

```
self._log(f"Running {self.n_steps} steps...")

# Reset client for clean run
client2 = UMAClient(cfg)
rng2    = np.random.default_rng(self.seed + 1)
psi0b   = 0.3*(rng2.standard_normal((N,N))+1j*rng2.standard_normal((N,N))
client2.initialize(psi0b, sigma0=0.05, dt=self.dt)

# Initialize semantic friction manually for step-by-step control
from uma.semantic.friction import SemanticFriction, FrictionRecord
# Build hamiltonian_fn: z -> H[psi] — connects friction to GR
def hamiltonian_fn(z: np.ndarray) -> float:
    psi_l = client2.projection.lift(z)
    H = client2.dynamics.hamiltonian(psi_l)
    return float(H.real if hasattr(H, 'real') else H)

# Tensor bridge — live T_μν ↔ G_μν tracking
tensor_bridge = TensorBridge(cfg, residual_threshold=0.15, window=30)

friction = SemanticFriction(z_target, friction_cfg,
                            hamiltonian_fn=hamiltonian_fn)
friction.reset(client2.posterior_state)

H_traj   = []
S_traj   = []
z_traj   = []
E_proxy  = []
pend_samples   = []
interf_amps    = []
t = 0.0

for step in range(self.n_steps):
    # 1. GENERIC step — use actual H[ψ]
    psi = client2.projection.lift(client2.posterior_state.mean)
    psi = client2.dynamics.step(psi, self.dt, rng=rng2)

    # Record actual Hamiltonian (not proxy)
    H_val = float(client2.dynamics.hamiltonian(psi).real
                  if hasattr(client2.dynamics.hamiltonian(psi), 'real')
                  else client2.dynamics.hamiltonian(psi))
    S_val = float(client2.dynamics.entropy(psi).real
                  if hasattr(client2.dynamics.entropy(psi), 'real')
                  else client2.dynamics.entropy(psi))
    H_traj.append(H_val)
    S_traj.append(S_val)

    # 2. MSR response field — psi_hat = -dH/dψ* (exact)
```

```
psi_hat = -client2.dynamics.dH_dpsi_conj(psi)

# 3. Interference amplitude (Planck × cos)
A = self._interference_amplitude(t, geo, field_scale)
interf_amps.append(A)

# 4. Sphere Venturi: inject both domains
phi_inject = psi_hat + A * np.exp(
    1j * 2 * math.pi * rng2.random((N,N))
)
psi = sphere_v.apply(psi, phi_inject, self.dt)

# 5. Project to z-space
z = client2.projection.project(psi)
z_traj.append(z.copy())
E_proxy.append(float(np.real(z @ z) / 2.0))

# 6. Pendulum observation → filter update
obs, y_obs = make_obs(t)
post, _ = client2.filter.update(
    FieldPosterior.full(z, client2.posterior_state.Sigma),
    obs, y_obs
)
client2.posterior_state = post

# 7. Pendulum sample (field measurement)
pend_samples.append(pendulum.sample_field(sphere_field, t))

# 8. Tensor bridge — T_μν ↔ G_μν live residual
t_rec = tensor_bridge.update(psi, psi_hat, t)

# 9. Semantic friction update
rec = friction.update(post, t)

t += self.dt

if self.verbose and step % max(1, self.n_steps//8) == 0:
    print(f"    step={step:4d}  H={H_val:.4e}  "
          f"S={S_val:.3f}  "
          f"friction={rec.friction:.3f}  "
          f"|dH/dt|={abs(rec.dH_dt):.3e}  "
          f"floor={friction_cfg.convergence_dz_dt:.3e}  "
          f"closed={rec.closed}")

t_satisfied = t_rec.einstein_satisfied
if rec.closed and t_satisfied:
    self._log(f"Closure (Ω) at step {step} — friction=0 AND Einstein
```

```
                break
            elif rec.closed and not t_satisfied:
                self._log(f"  step={step}: friction closed, waiting for Einstein

        # ── 8. Assemble result ──────────────────────────────────────────
        fw = [r.friction for r in friction.records]

        tensor_summary = tensor_bridge.summary()
        return UMAPipelineResult(
            geometry=geo,
            H_trajectory=H_traj,
            S_trajectory=S_traj,
            z_trajectory=z_traj,
            E_proxy=E_proxy,
            pendulum_samples=pend_samples,
            friction_walk=fw,
            is_closed=friction.is_closed,
            closure_step=friction.closure_step,
            stage=("solve" if friction.is_closed else
                   "inarticulation" if fw and fw[-1] < 0.20 else
                   "stability" if fw and fw[-1] < 0.50 else "ingestion"),
            msr_cosine=msr_cos,
            msr_verified=msr_ok,
            rg_basin=rg_basin,
            rg_z=rg_z,
            interference_amplitudes=interf_amps,
            tensor_records=tensor_bridge.records,
            n_steps=len(H_traj),
            dt=self.dt,
        )


# ---------------------------------------------------------------------------
# Entry point
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    print()
    pipe   = UMAPipeline(L=1.0, n_steps=300, dt=0.04, verbose=True)
    result = pipe.run()
    print()
    result.report()
```